

METRC Integration

Technical Guide

Audience: Developers & Integration Engineers

ERPNext v15 · cannabis_management

California Metrc Web API v2

4 August 2026

Metric Integration Guide — `cannabis_management`

Target: California Metric Web API v2 **App:** `cannabis_management` (Frappe/ERPNext v15) **Status of this document:** written 2026-08-03, against the 605-page CA Web API spec and live sandbox probing.

Table of contents

1. Where you stand today
 2. API fundamentals you must design around
 3. Domain model mapping
 4. Target architecture
 5. New doctypes
 6. The HTTP client layer
 7. The pull (inbound sync) engine
 8. The push (outbound) engine
 9. Flow A — Distribution & sales
 10. Flow B — Manufacturing / processing
 11. Flow C — Cultivation
 12. Reconciliation & variance reporting
 13. Hooks wiring
 14. Testing against the sandbox
 15. Production go-live checklist
 16. Security
 17. Appendix A — Endpoint reference
 18. Appendix B — Payload schemas
-

1. Where you stand today

Verified live (2026-08-03)

Check	Result
<code>sandbox-api-ca.metroc.com</code> + integrator key + CA sandbox user key	200 — 20 facilities returned
<code>api-ca.metroc.com</code> (production) + same integrator key	401 — not yet approved
GET <code>/packages/v2/active</code> on <code>C12-1000001-LIC</code>	200 — real seeded packages
GET <code>/tags/v2/package/available</code>	200 — bare array (not paginated)
GET <code>/unitsofmeasure/v2/active</code>	200 — 11 UOMs
GET <code>/employees/v2/permissions</code>	404 — not deployed in CA

Sandbox facilities available to you span every license type you need: microbusiness (`C12-1000001-LIC` ... `-1000010-LIC`), cultivator (`CML17-0000001`), distributor (`C11-0000090-LIC`), retailer (`M10-0000004-LIC`), manufacturer (`CDPH-0000003`), transport (`C13-0000092-LIC`), and two labs.

What already exists in your app

You are further along than a greenfield integration. These are already in the codebase and should be **reused, not recreated**:

Existing artifact

`Metric Tag` doctype (`tag_code` , `muid` , `status` , `item_code` , `current_qty` , `uom` , `warehouse` , `last_transaction_type` ,

`Warehouse.custom_metrc_license_number` , `Warehouse.custom_license_type`

`Batch.custom_metrc_tag` , `Batch.custom_metrc_last_synced` , `Batch.custom_metrc_license_source` , `Batch.custom_stra`
`Batch.custom_procurement_status`

`Project.custom_metrc_tag`

`Purchase Receipt.custom_metrc_tag_original`

`Work Order.custom_customer_material_batch` (labelled "Customer Material METRC Tag")

`Stock Reconciliation.custom_metrc_correction_made` , `custom_variance_reason` , `custom_compliance_notes`

`overrides/license_compliance.py`

`si_manifest_cdt`

Existing artifact

`requests` 2.34.2 in the bench venv

The single most important consequence: your domain model is already `Warehouse = Facility`, `Batch = Package`, `Metric Tag = tag`. Do not introduce a parallel "Metric Package" doctype. Sync into what exists.

2. API fundamentals you must design around

These are not trivia. Each one forces a specific architectural decision.

2.1 Authentication

HTTP Basic. **Username = integrator (software) key. Password = user key.**

```
Authorization: Basic base64(integrator_key + ":" + user_key)
```

The integrator key is per-software-vendor and company-wide. The user key is per-Metric-user and carries that user's permissions — everything your software can do is the intersection of the two. A user key is *not* per-facility, but the facilities it can reach are determined by that user's account.

2.2 Environments

Environment	Host
CA Sandbox	https://sandbox-api-ca.metric.com
CA Production	https://api-ca.metric.com

Production requires a **separate Metric integrator validation** — you demo your sandbox integration to Metric and they enable the key. Budget calendar time for this; it is not a config change.

2.3 Dates and times — three distinct rules

1. **All dates/times are ISO 8601.** Date-only fields accept `YYYY-MM-DD` only (`YYYY-DDD` ordinal dates are rejected).
2. **In query strings, + must be percent-encoded as %2B .**
`lastModifiedStart=2023-11-01T08:00:00+02:00` silently returns wrong results; `...%2B02:00` is correct. Use `urlencode` on every param — never string-concatenate.
3. **SalesDateTime on sales receipts must be facility-local wall-clock time with no timezone suffix.** If the facility is in Pacific, send Pacific time. This is the opposite of every other field, and it is the single most common cause of rejected receipts.

2.4 Object limiting — max 10 objects per request

Any POST/PUT/DELETE that accepts an array is capped at **10 objects**. Exceeding it returns **HTTP 413**. Your write layer must chunk unconditionally.

2.5 Rate limiting — per facility licence

Limits are per facility licence (per API key for the few licence-less endpoints). Exceeding returns **429**, sometimes with a `Retry-After` header giving seconds to wait. The exact limits depend on your Metric contract tier.

Corollary: batch aggressively. One POST with 10 receipts, not 10 POSTs with one each.

2.6 The `lastModified` window — this shapes your whole sync design

Metric exposes `LastModified` on most entities and **requires** a bounded `lastModifiedStart` / `lastModifiedEnd` range on list endpoints. You cannot request "everything".

The spec is explicit about direction: **poll forward in chronological order, oldest to newest.**

`LastModified` only ever moves forward. If you poll newest-first, a record modified *during* your sweep moves into a window you have already read, and you lose it — possibly until the next full cycle.

This mandates a **persisted per-(licence, endpoint) cursor**. That's the `Metrc Sync State` doctype in §5.

2.7 Two response shapes

When you pass `pageSize`, list endpoints wrap the payload:

```
{ "Data": [...], "Total": 2, "TotalRecords": 2, "PageSize": 2,
  "RecordsOnPage": 2, "Page": 1, "CurrentPage": 1, "TotalPages": 1 }
```

Without `pageSize` — and unconditionally on some endpoints (`/tags/v2/package/available` , `/unitsofmeasure/v2/active`) — you get a **bare JSON array**. Both shapes confirmed live. Your client must normalise this in one place. `pageSize` is capped at 20 on many endpoints.

2.8 POST responses return IDs in order

```
{ "Ids": [1, 2, 3], "Warnings": null }
```

The IDs correspond positionally to the objects you submitted. This is how you map created Metrc entities back to ERPNext rows. **Many PUT endpoints return an empty body** ("No response" in the docs) — do not try to parse them.

2.9 HTTP status codes

Code	Meaning	Your handling
200	OK	Parse body (may be empty on PUT)
401	Bad/unauthorised keys, or wrong environment	Fail loud, do not retry
403	Sandbox-only endpoint called on prod	Fail loud
404	Endpoint not deployed in this state	Fail loud, log
413	More than 10 objects in the array	Bug in your chunker — fail loud
429	Rate limited	Honour <code>Retry-After</code> , exponential backoff
500	Metrc-side error; message usually in body	Retry with backoff, then park

2.10 Record matching by name

Harvests, plant batches, locations, items and strains can be referenced by **name instead of ID**, case-insensitively. This is very useful — it means you can push an ERPNext Item name directly as `"Item": "Buds"` without first resolving a Metrc item ID. It also means **name collisions are silent data corruption**, so enforce uniqueness on your side.

2.11 Use v2 only

Every module exists in v1 and v2. v2 adds pagination, `inactive` listings, sublocations, `/tags` , retail ID, and `Ids` in POST responses. There is no reason to touch v1 on a new build.

2.12 Webhooks exist but are contract-gated

`PUT /webhooks/v2` , `PUT /webhooks/v2/enable/{id}` , `DELETE /webhooks/v2/{id}` are listed, but the spec says availability depends on your service tier. The detail pages are not in the CA printable doc. **Design for polling; treat webhooks as a later optimisation** that reduces poll frequency, not as the primary transport.

3. Domain model mapping

3.1 Core entities

Metrc	ERPNext / <code>cannabis_management</code>	Join key
Facility / Licence	Warehouse	<code>Warehouse.custom_metrc_license_number</code>
Package	Batch	<code>Batch.custom_metrc_tag</code>
Package tag (unused/active/empty)	Metric Tag	<code>Metric Tag.tag_code</code>
Item	Item	<code>Item.custom_metrc_item_name</code> (to add)
Item Category	Item Group	mapping table
Strain	Strain (via <code>Batch.custom_strain_name</code>)	name
Location / Sublocation	Warehouse child / storage bin	mapping table
Unit of Measure	UOM	mapping table (Metrc has 11 fixed)
Sales Receipt	Sales Invoice	<code>Sales Invoice.custom_metrc_receipt_id</code> (to add)
Outgoing Transfer	Delivery Note + <code>si_manifest_cdt</code>	<code>custom_metrc_transfer_id</code> (to add)
Incoming Transfer	Purchase Receipt	<code>custom_metrc_tag_original</code>
Processing Job	Conversion Entry / Manufacture Stock Entry / Work Order	<code>custom_metrc_job_id</code> (to add)
Harvest	Production Batch / <code>farm_production_batch</code>	<code>custom_metrc_harvest_id</code> (to add)
Plant Batch	Cloning Batch	<code>custom_metrc_plantbatch_id</code> (to add)
Package Adjustment	Stock Reconciliation	<code>custom_metrc_correction_made</code>
Lab Test Result	<code>lab_batch_entry</code> / <code>all_lab_tolling_data</code>	<code>custom_metrc_labtest_id</code> (to add)

3.2 Units of measure — the 11 Metrc UOMs

Confirmed live from `GET /unitsofmeasure/v2/active` :

Name	Abbrev	Quantity type
Each	ea	CountBased
Grams	g	WeightBased
Kilograms	kg	WeightBased
Milligrams	mg	WeightBased
Ounces	oz	WeightBased
Pounds	lb	WeightBased
Fluid Ounces	fl oz	VolumeBased
Gallons	gal	VolumeBased
Liters	l	VolumeBased
Milliliters	ml	VolumeBased
Pints	pt	VolumeBased

Rule: every ERPNext UOM used on a Metrc-tracked Item must map to exactly one of these. Enforce it at validation time, not at push time — a rejected push at 2am is much worse than a blocked save.

Rule: never convert units yourself. Send the quantity in the UOM Metrc holds for that package. Rounding drift between g/oz is a compliance discrepancy.

3.3 The tag lifecycle

```
Metrc: tag ordered → received → available → assigned to package → package active → finished/
discontinued
App:      —           —           Metric Tag.status=Unused → Active (Batch created) → Empty
```

GET `/tags/v2/package/available` and `/tags/v2/plant/available` give you the unused pool. Sync these into `Metric Tag` with `status=Unused`. When you create a package you consume one; when a package is finished, set `status=Empty`.

Never generate tag codes. They are physical labels ordered from Metrc. In sandbox only, `POST /sandbox/v2/facility/tags` mints up to 1,000 at a time.

4. Target architecture

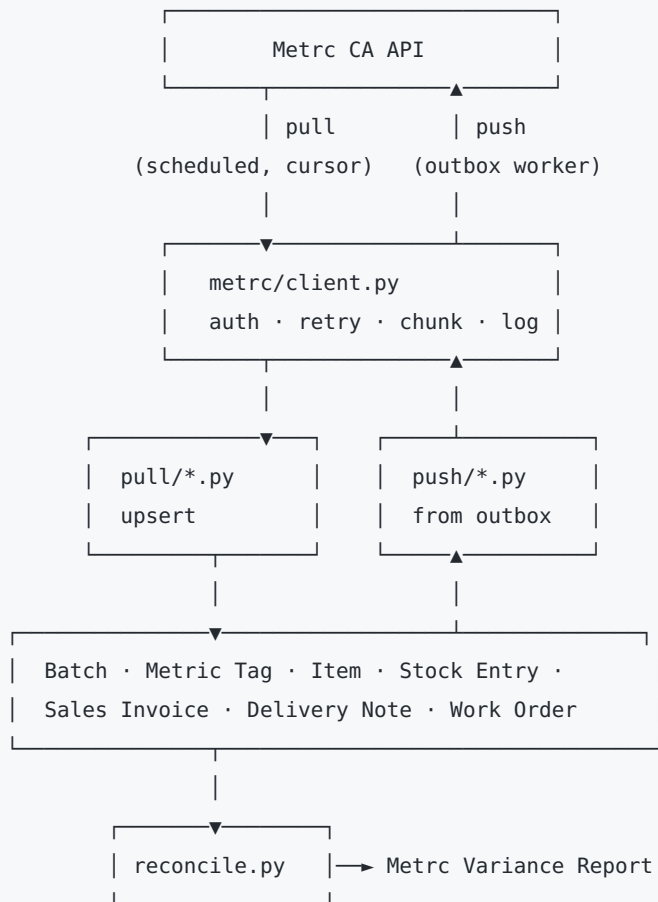
4.1 Design principles

1. **Metrc is the system of record for compliance state.** Where they disagree, Metrc wins and the difference becomes a variance to investigate — never a silent overwrite in either direction.
2. **Pull and push are separate subsystems** with separate failure domains. A broken push must never stall the pull.
3. **All writes go through an outbox.** Never call Metrc synchronously from a document hook. A Metrc 500 must not roll back a Sales Invoice submission.
4. **Every request/response is logged.** State regulators ask for audit trails, and you will need them to debug rejections.
5. **Idempotency everywhere.** Retries are guaranteed; duplicate packages in a state system are a compliance incident.

4.2 Module layout

```
cannabis_management/  
└─ metrc/  
  │ └─ __init__.py  
  │ └─ client.py          # HTTP transport: auth, retry, chunking, shape normalisation  
  │ └─ exceptions.py      # MetrcError hierarchy  
  │ └─ config.py          # settings accessor + licence resolution  
  │ └─ mapping.py         # UOM / item-category / location translation  
  │ └─ pull/  
  │   │ └─ __init__.py  
  │   │ └─ base.py        # cursor-window sweep driver  
  │   │ └─ packages.py  
  │   │ └─ items.py  
  │   │ └─ strains.py  
  │   │ └─ tags.py  
  │   │ └─ transfers.py  
  │   │ └─ sales.py  
  │   │ └─ harvests.py  
  │   │ └─ plantbatches.py  
  │   └─ labtests.py  
  │ └─ push/  
  │   │ └─ __init__.py  
  │   │ └─ outbox.py      # enqueue + worker  
  │   │ └─ packages.py  
  │   │ └─ sales.py  
  │   │ └─ transfers.py  
  │   └─ processing.py  
  │ └─ reconcile.py       # variance detection  
  └─ doctype/  
    │ └─ metrc_settings/  
    │ └─ metrc_facility/      (child table)  
    │ └─ metrc_sync_state/  
    │ └─ metrc_api_log/  
    │ └─ metrc_outbox/  
    └─ metrc_uom_map/        (child table)
```

4.3 Data flow



5. New doctypes

Developer mode is already on for `stage.alltechvirtual.com`, so creating these writes the JSON into the app automatically.

5.1 Metrc Settings (Single)

Field	Type	Notes
<code>enabled</code>	Check	Master kill switch
<code>environment</code>	Select	<code>Sandbox</code> / <code>Production</code>
<code>sandbox_base_url</code>	Data	<code>https://sandbox-api-ca.metrc.com</code>
<code>production_base_url</code>	Data	<code>https://api-ca.metrc.com</code>
<code>integrator_key</code>	Password	Software/vendor key
<code>facilities</code>	Table → Metrc Facility	Per-licence config
<code>uom_map</code>	Table → Metrc UOM Map	ERPNext UOM ↔ Metrc UOM
<code>default_page_size</code>	Int	Default <code>20</code>
<code>window_hours</code>	Int	<code>lastModified</code> window width, default <code>24</code>
<code>max_retries</code>	Int	Default <code>4</code>
<code>push_enabled</code>	Check	Separate switch — pull-only mode for phase 1
<code>dry_run</code>	Check	Log the payload, do not transmit
<code>log_retention_days</code>	Int	Default <code>120</code>
<code>alert_email</code>	Data	Where sync failures go

`dry_run` and `push_enabled` are not optional niceties. They are how you validate the push layer against production data without transmitting to a state system.

5.2 Metrc Facility (child table)

Field	Type	Notes
<code>license_number</code>	Data (reqd)	e.g. <code>C12-1000001-LIC</code>
<code>facility_name</code>	Data	From <code>GET / facilities/v2/</code>
<code>warehouse</code>	Link → Warehouse	The ERPNext side
<code>user_key</code>	Password	Per-facility Metrc user key
<code>facility_timezone</code>	Select	Default <code>America/Los_Angeles</code> — drives <code>SalesDateTime</code>
<code>is_active</code>	Check	
<code>sync_packages</code> / <code>sync_sales</code> / <code>sync_transfers</code> / <code>sync_plants</code> / <code>sync_harvests</code>	Check	Per-facility feature flags

Per-facility feature flags matter because a retailer facility has no plants and a cultivator has no sales receipts. Polling endpoints a licence type cannot use just burns rate limit.

5.3 Metrc Sync State

Naming: `{license_number}::{endpoint_key}`

Field	Type	Notes
<code>license_number</code>	Data	
<code>endpoint_key</code>	Data	e.g. <code>packages.active</code>
<code>cursor_last_modified</code>	Datetime	The forward-only watermark
<code>last_run_start</code> / <code>last_run_end</code>	Datetime	
<code>last_status</code>	Select	<code>Success</code> / <code>Partial</code> / <code>Failed</code>
<code>last_error</code>	Small Text	
<code>records_synced</code>	Int	Cumulative
<code>consecutive_failures</code>	Int	Drives alerting

5.4 Metrc API Log

Field	Type
timestamp	Datetime
direction	Select Pull / Push
method	Data
endpoint	Small Text
license_number	Data
request_body	Code (JSON)
response_status	Int
response_body	Code (JSON)
duration_ms	Int
error	Small Text
reference_doctype / reference_name	Link / Dynamic Link

Redact the keys before writing. They live in the `Authorization` header, so simply never log headers.

5.5 Metrc Outbox

Field	Type	Notes
status	Select	Queued / In Progress / Success / Failed / Parked
operation	Data	e.g. <code>packages.create</code>
license_number	Data	
payload	Code (JSON)	The exact object to send
idempotency_key	Data (unique)	See §8.2
reference_doctype / reference_name	Link / Dynamic Link	Source document
attempts	Int	
next_attempt_at	Datetime	Backoff schedule
last_error	Small Text	
metrc_id	Data	ID returned on success
response	Code (JSON)	

5.6 Bootstrap script

```

# bench --site stage.alltechvirtual.com execute \
#   cannabis_management.metr.c.install.create_doctypes

import frappe

MODULE = "Cannabis Management"

def _mk(name, fields, **kw):
    if frappe.db.exists("DocType", name):
        return
    doc = frappe.get_doc({
        "doctype": "DocType",
        "name": name,
        "module": MODULE,
        "custom": 0,
        "fields": fields,
        "permissions": [{
            "role": "System Manager",
            "read": 1, "write": 1, "create": 1, "delete": 1,
        }],
        **kw,
    })
    doc.insert(ignore_permissions=True)

def create_doctypes():
    _mk("Metrc UOM Map", [
        {"fieldname": "erpNext_uom", "fieldtype": "Link", "options": "UOM",
         "label": "ERPNext UOM", "in_list_view": 1, "reqd": 1},
        {"fieldname": "metrc_uom", "fieldtype": "Select", "label": "Metrc UOM",
         "in_list_view": 1, "reqd": 1,
         "options": "Each\nGrams\nKilograms\nMilligrams\nOunces\nPounds\n"
                    "Fluid Ounces\nGallons\nLiters\nMilliliters\nPints"},
    ], istable=1)

    _mk("Metrc Facility", [
        {"fieldname": "license_number", "fieldtype": "Data",
         "label": "License Number", "in_list_view": 1, "reqd": 1},
        {"fieldname": "facility_name", "fieldtype": "Data",
         "label": "Facility Name", "in_list_view": 1},
        {"fieldname": "warehouse", "fieldtype": "Link", "options": "Warehouse",
         "label": "Warehouse", "in_list_view": 1},
        {"fieldname": "user_key", "fieldtype": "Password", "label": "User API Key"},
        {"fieldname": "facility_timezone", "fieldtype": "Data",
         "label": "Facility Timezone", "default": "America/Los_Angeles"},
        {"fieldname": "is_active", "fieldtype": "Check",
         "label": "Active", "default": "1"},
        {"fieldname": "sync_packages", "fieldtype": "Check",
         "label": "Sync Packages", "default": "1"},
    ])

```

```

{"fieldname": "sync_sales", "fieldtype": "Check", "label": "Sync Sales"},
{"fieldname": "sync_transfers", "fieldtype": "Check",
  "label": "Sync Transfers", "default": "1"},
{"fieldname": "sync_plants", "fieldtype": "Check", "label": "Sync Plants"},
{"fieldname": "sync_harvests", "fieldtype": "Check", "label": "Sync Harvests"},
], istable=1)

_mk("Metrc Settings", [
  {"fieldname": "enabled", "fieldtype": "Check", "label": "Enabled"},
  {"fieldname": "environment", "fieldtype": "Select", "label": "Environment",
    "options": "Sandbox\nProduction", "default": "Sandbox", "reqd": 1},
  {"fieldname": "sandbox_base_url", "fieldtype": "Data",
    "label": "Sandbox Base URL",
    "default": "https://sandbox-api-ca.metrc.com"},
  {"fieldname": "production_base_url", "fieldtype": "Data",
    "label": "Production Base URL", "default": "https://api-ca.metrc.com"},
  {"fieldname": "integrator_key", "fieldtype": "Password",
    "label": "Integrator (Software) API Key"},
  {"fieldname": "sb_1", "fieldtype": "Section Break", "label": "Facilities"},
  {"fieldname": "facilities", "fieldtype": "Table",
    "options": "Metrc Facility", "label": "Facilities"},
  {"fieldname": "sb_2", "fieldtype": "Section Break", "label": "UOM Mapping"},
  {"fieldname": "uom_map", "fieldtype": "Table",
    "options": "Metrc UOM Map", "label": "UOM Map"},
  {"fieldname": "sb_3", "fieldtype": "Section Break", "label": "Behaviour"},
  {"fieldname": "default_page_size", "fieldtype": "Int",
    "label": "Default Page Size", "default": "20"},
  {"fieldname": "window_hours", "fieldtype": "Int",
    "label": "Sync Window (hours)", "default": "24"},
  {"fieldname": "max_retries", "fieldtype": "Int",
    "label": "Max Retries", "default": "4"},
  {"fieldname": "push_enabled", "fieldtype": "Check", "label": "Enable Push"},
  {"fieldname": "dry_run", "fieldtype": "Check", "label": "Dry Run (log only)"},
  {"fieldname": "log_retention_days", "fieldtype": "Int",
    "label": "Log Retention (days)", "default": "120"},
  {"fieldname": "alert_email", "fieldtype": "Data", "label": "Alert Email"},
], issingle=1)

_mk("Metrc Sync State", [
  {"fieldname": "license_number", "fieldtype": "Data",
    "label": "License Number", "in_list_view": 1, "reqd": 1},
  {"fieldname": "endpoint_key", "fieldtype": "Data",
    "label": "Endpoint Key", "in_list_view": 1, "reqd": 1},
  {"fieldname": "cursor_last_modified", "fieldtype": "Datetime",
    "label": "Cursor (LastModified)", "in_list_view": 1},
  {"fieldname": "last_run_start", "fieldtype": "Datetime", "label": "Last Run Start"},
  {"fieldname": "last_run_end", "fieldtype": "Datetime", "label": "Last Run End"},
  {"fieldname": "last_status", "fieldtype": "Select", "label": "Last Status",
    "options": "\nSuccess\nPartial\nFailed", "in_list_view": 1},
  {"fieldname": "last_error", "fieldtype": "Small Text", "label": "Last Error"},

```

```

        {"fieldname": "records_synced", "fieldtype": "Int", "label": "Records Synced"},
        {"fieldname": "consecutive_failures", "fieldtype": "Int",
         "label": "Consecutive Failures"},
    ], autoname="prompt")

_mk("Metrc API Log", [
    {"fieldname": "timestamp", "fieldtype": "Datetime",
     "label": "Timestamp", "in_list_view": 1},
    {"fieldname": "direction", "fieldtype": "Select", "label": "Direction",
     "options": "Pull\nPush", "in_list_view": 1},
    {"fieldname": "method", "fieldtype": "Data", "label": "Method"},
    {"fieldname": "endpoint", "fieldtype": "Small Text",
     "label": "Endpoint", "in_list_view": 1},
    {"fieldname": "license_number", "fieldtype": "Data", "label": "License Number"},
    {"fieldname": "request_body", "fieldtype": "Code",
     "options": "JSON", "label": "Request Body"},
    {"fieldname": "response_status", "fieldtype": "Int",
     "label": "Status", "in_list_view": 1},
    {"fieldname": "response_body", "fieldtype": "Code",
     "options": "JSON", "label": "Response Body"},
    {"fieldname": "duration_ms", "fieldtype": "Int", "label": "Duration (ms)"},
    {"fieldname": "error", "fieldtype": "Small Text", "label": "Error"},
    {"fieldname": "reference_doctype", "fieldtype": "Link",
     "options": "DocType", "label": "Reference DocType"},
    {"fieldname": "reference_name", "fieldtype": "Dynamic Link",
     "options": "reference_doctype", "label": "Reference Name"},
    ], autoname="hash")

_mk("Metrc Outbox", [
    {"fieldname": "status", "fieldtype": "Select", "label": "Status",
     "options": "Queued\nIn Progress\nSuccess\nFailed\nParked",
     "default": "Queued", "in_list_view": 1},
    {"fieldname": "operation", "fieldtype": "Data",
     "label": "Operation", "in_list_view": 1, "reqd": 1},
    {"fieldname": "license_number", "fieldtype": "Data",
     "label": "License Number", "in_list_view": 1, "reqd": 1},
    {"fieldname": "payload", "fieldtype": "Code",
     "options": "JSON", "label": "Payload"},
    {"fieldname": "idempotency_key", "fieldtype": "Data",
     "label": "Idempotency Key", "unique": 1},
    {"fieldname": "reference_doctype", "fieldtype": "Link",
     "options": "DocType", "label": "Reference DocType"},
    {"fieldname": "reference_name", "fieldtype": "Dynamic Link",
     "options": "reference_doctype", "label": "Reference Name"},
    {"fieldname": "attempts", "fieldtype": "Int", "label": "Attempts"},
    {"fieldname": "next_attempt_at", "fieldtype": "Datetime",
     "label": "Next Attempt At"},
    {"fieldname": "last_error", "fieldtype": "Small Text", "label": "Last Error"},
    {"fieldname": "metrc_id", "fieldtype": "Data", "label": "Metrc ID"},
    {"fieldname": "response", "fieldtype": "Code",

```

```
        "options": "JSON", "label": "Response"},  
    ], autoname="hash")  
  
    frappe.db.commit()
```

5.7 Custom fields to add

```
# cannabis_management/metrc/install.py (continued)

from frappe.custom.doctype.custom_field.custom_field import create_custom_fields

def install_metrc_fields():
    create_custom_fields({
        "Item": [
            {"fieldname": "custom_metrc_section", "fieldtype": "Section Break",
             "label": "METRC", "insert_after": "item_group", "collapsible": 1},
            {"fieldname": "custom_metrc_tracked", "fieldtype": "Check",
             "label": "METRC Tracked", "insert_after": "custom_metrc_section"},
            {"fieldname": "custom_metrc_item_name", "fieldtype": "Data",
             "label": "METRC Item Name", "insert_after": "custom_metrc_tracked",
             "description": "Exact Metrc item name. Used for name-based matching."},
            {"fieldname": "custom_metrc_item_id", "fieldtype": "Int",
             "label": "METRC Item ID", "insert_after": "custom_metrc_item_name",
             "read_only": 1},
            {"fieldname": "custom_metrc_category", "fieldtype": "Data",
             "label": "METRC Item Category", "insert_after": "custom_metrc_item_id"},
        ],
        "Sales Invoice": [
            {"fieldname": "custom_metrc_receipt_id", "fieldtype": "Data",
             "label": "METRC Receipt ID", "read_only": 1,
             "insert_after": "customer", "allow_on_submit": 1},
            {"fieldname": "custom_metrc_sync_status", "fieldtype": "Select",
             "label": "METRC Sync Status",
             "options": "\nNot Required\nQueued\nSynced\nFailed",
             "insert_after": "custom_metrc_receipt_id", "allow_on_submit": 1},
        ],
        "Delivery Note": [
            {"fieldname": "custom_metrc_transfer_id", "fieldtype": "Data",
             "label": "METRC Transfer ID", "read_only": 1,
             "insert_after": "customer", "allow_on_submit": 1},
            {"fieldname": "custom_metrc_manifest_number", "fieldtype": "Data",
             "label": "METRC Manifest #", "read_only": 1,
             "insert_after": "custom_metrc_transfer_id", "allow_on_submit": 1},
        ],
        "Work Order": [
            {"fieldname": "custom_metrc_job_id", "fieldtype": "Data",
             "label": "METRC Processing Job ID", "read_only": 1,
             "insert_after": "custom_customer_material_batch",
             "allow_on_submit": 1},
        ],
        "Batch": [
            {"fieldname": "custom_metrc_package_id", "fieldtype": "Int",
             "label": "METRC Package ID", "read_only": 1,
             "insert_after": "custom_metrc_tag"},
            {"fieldname": "custom_metrc_quantity", "fieldtype": "Float",
             "label": "METRC Quantity", "read_only": 1,
```

```
        "insert_after": "custom_metrc_package_id",
        "description": "Quantity per Metrc. Compare against ERPNext for variance."},
    {"fieldname": "custom_metrc_uom", "fieldtype": "Data",
     "label": "METRC UOM", "read_only": 1,
     "insert_after": "custom_metrc_quantity"},
    {"fieldname": "custom_metrc_status", "fieldtype": "Data",
     "label": "METRC Package Status", "read_only": 1,
     "insert_after": "custom_metrc_uom"},
    ],
})
```

6. The HTTP client layer

```
# cannabis_management/metrc/exceptions.py
"""Exception hierarchy for the Metrc integration."""

class MetrcError(Exception):
    """Base for all Metrc failures."""

    def __init__(self, message, status_code=None, body=None, endpoint=None):
        super().__init__(message)
        self.status_code = status_code
        self.body = body
        self.endpoint = endpoint

class MetrcAuthError(MetrcError):
    """401/403 – keys are wrong or the environment is wrong. Never retry."""

class MetrcRateLimitError(MetrcError):
    """429 – back off and retry."""

    def __init__(self, message, retry_after=None, **kw):
        super().__init__(message, **kw)
        self.retry_after = retry_after

class MetrcPayloadTooLargeError(MetrcError):
    """413 – more than 10 objects. A bug in our chunker. Never retry."""

class MetrcValidationError(MetrcError):
    """400 – Metrc rejected the data. Never retry; a human must fix it."""

class MetrcServerError(MetrcError):
    """500 – Metrc-side. Retry with backoff."""

class MetrcNotConfigured(MetrcError):
    """Settings missing or integration disabled."""
```

```

# cannabis_management/metrc/config.py
"""Settings accessor and licence resolution for the Metrc integration."""

import frappe

from cannabis_management.metrc.exceptions import MetrcNotConfigured

SETTINGS = "Metrc Settings"

def get_settings():
    return frappe.get_cached_doc(SETTINGS)

def is_enabled():
    return bool(frappe.db.get_single_value(SETTINGS, "enabled"))

def base_url():
    s = get_settings()
    url = (s.production_base_url if s.environment == "Production"
           else s.sandbox_base_url)
    if not url:
        raise MetrcNotConfigured(f"No base URL set for environment {s.environment}")
    return url.rstrip("/")

def integrator_key():
    key = get_settings().get_password("integrator_key", raise_exception=False)
    if not key:
        raise MetrcNotConfigured("Integrator API key is not set in Metrc Settings")
    return key

def get_facility(license_number):
    """Return the Metrc Facility child row for a licence, or raise."""
    for row in get_settings().facilities:
        if row.license_number == license_number:
            return row
    raise MetrcNotConfigured(f"License {license_number} is not configured")

def user_key(license_number):
    row = get_facility(license_number)
    key = row.get_password("user_key", raise_exception=False)
    if not key:
        raise MetrcNotConfigured(f"No user key configured for {license_number}")
    return key

```

```

def active_facilities(feature=None):
    """Active facilities, optionally filtered by a sync_* feature flag."""
    out = []
    for row in get_settings().facilities:
        if not row.is_active:
            continue
        if feature and not row.get(feature):
            continue
        out.append(row)
    return out

def warehouse_for_license(license_number):
    row = get_facility(license_number)
    if row.warehouse:
        return row.warehouse
    return frappe.db.get_value(
        "Warehouse", {"custom_metrc_license_number": license_number}, "name"
    )

def license_for_warehouse(warehouse):
    return frappe.db.get_value(
        "Warehouse", warehouse, "custom_metrc_license_number"
    )

```

```

# cannabis_management/metrc/client.py
"""
Metrc Web API v2 transport layer.

Responsibilities:
* HTTP Basic auth (integrator key : user key)
* Correct query-string encoding (the "+" -> %2B trap on timestamps)
* Response-shape normalisation (paginated envelope vs bare array)
* Automatic pagination
* Object limiting (max 10 objects per write) via chunking
* Retry with backoff, honouring Retry-After on 429
* Full request/response logging to Metrc API Log
"""

import json
import time
from urllib.parse import urlencode

import frappe
import requests
from frappe.utils import now_datetime

from cannabis_management.metrc import config
from cannabis_management.metrc.exceptions import (
    MetrcAuthError,
    MetrcError,
    MetrcNotConfigured,
    MetrcPayloadTooLargeError,
    MetrcRateLimitError,
    MetrcServerError,
    MetrcValidationError,
)

# Hard cap imposed by Metrc on arrays in request bodies. Exceeding it -> HTTP 413.
MAX_OBJECTS_PER_REQUEST = 10

# Metrc caps pageSize at 20 on many endpoints.
MAX_PAGE_SIZE = 20

DEFAULT_TIMEOUT = 60
RETRYABLE_STATUS = {429, 500, 502, 503, 504}

class MetrcClient:
    """One client per facility licence."""

    def __init__(self, license_number=None, timeout=DEFAULT_TIMEOUT):
        if not config.is_enabled():
            raise MetrcNotConfigured("Metrc integration is disabled")

```

```

self.license_number = license_number
self.base_url = config.base_url()
self.timeout = timeout
self.settings = config.get_settings()
self.max_retries = self.settings.max_retries or 4

self.session = requests.Session()
self.session.auth = (
    config.integrator_key(),
    config.user_key(license_number) if license_number else "",
)
self.session.headers.update({
    "Content-Type": "application/json",
    "Accept": "application/json",
})

# ----- core

def request(self, method, path, params=None, body=None,
            reference=None, _direction=None):
    """
    Issue a single request. Returns the parsed body, or None for the
    many PUT endpoints that respond with no content.
    """
    params = dict(params or {})
    if self.license_number and "licenseNumber" not in params:
        params["licenseNumber"] = self.license_number

    # urlencode handles the "+ must be %2B" requirement on timestamps.
    # Never build query strings by concatenation.
    qs = urlencode({k: v for k, v in params.items() if v is not None})
    url = f"{self.base_url}{path}" + (f"?{qs}" if qs else "")

    direction = _direction or ("Pull" if method == "GET" else "Push")

    if body is not None and isinstance(body, list):
        if len(body) > MAX_OBJECTS_PER_REQUEST:
            raise MetrcPayloadTooLargeError(
                f"{len(body)} objects exceeds the Metrc limit of "
                f"{MAX_OBJECTS_PER_REQUEST}. Use post_chunked()."
            )

    if self.settings.dry_run and method != "GET":
        self._log(direction, method, url, body, 0, None,
                  0, "DRY RUN – not transmitted", reference)
        return None

    last_exc = None
    for attempt in range(self.max_retries + 1):
        started = time.monotonic()

```

```

try:
    resp = self.session.request(
        method, url,
        data=json.dumps(body) if body is not None else None,
        timeout=self.timeout,
    )
except requests.RequestException as e:
    last_exc = MetrcServerError(f"Network error: {e}", endpoint=url)
    self._log(direction, method, url, body, 0, None,
               int((time.monotonic() - started) * 1000), str(e), reference)
    self._sleep_backoff(attempt)
    continue

duration_ms = int((time.monotonic() - started) * 1000)
text = resp.text or ""

try:
    parsed = json.loads(text) if text.strip() else None
except ValueError:
    parsed = text

self._log(direction, method, url, body, resp.status_code,
           parsed, duration_ms, None, reference)

if resp.status_code < 300:
    return parsed

exc = self._to_exception(resp, parsed, url)

if resp.status_code not in RETRYABLE_STATUS:
    raise exc

last_exc = exc
if attempt >= self.max_retries:
    break

if isinstance(exc, MetrcRateLimitError) and exc.retry_after:
    time.sleep(min(exc.retry_after, 300))
else:
    self._sleep_backoff(attempt)

raise last_exc or MetrcError("Request failed", endpoint=url)

@staticmethod
def _to_exception(resp, parsed, url):
    msg = ""
    if isinstance(parsed, dict):
        msg = parsed.get("Message") or parsed.get("message") or ""
    msg = msg or (resp.text or "")[:500]

```

```

kw = {"status_code": resp.status_code, "body": parsed, "endpoint": url}

if resp.status_code in (401, 403):
    return MetrcAuthError(f"Auth failed ({resp.status_code}): {msg}", **kw)
if resp.status_code == 413:
    return MetrcPayloadTooLargeError(f"Payload too large: {msg}", **kw)
if resp.status_code == 400:
    return MetrcValidationError(f"Validation failed: {msg}", **kw)
if resp.status_code == 429:
    ra = resp.headers.get("Retry-After")
    try:
        ra = int(ra) if ra else None
    except ValueError:
        ra = None
    return MetrcRateLimitError(f"Rate limited: {msg}", retry_after=ra, **kw)
if resp.status_code >= 500:
    return MetrcServerError(f"Metrc server error: {msg}", **kw)
return MetrcError(f"HTTP {resp.status_code}: {msg}", **kw)

@staticmethod
def _sleep_backoff(attempt):
    # 1s, 2s, 4s, 8s ... capped.
    time.sleep(min(2 ** attempt, 60))

# ----- read API

@staticmethod
def _unwrap(payload):
    """
    Normalise the two response shapes.

    Paginated: {"Data": [...], "TotalPages": n, "Page": p, ...}
    Bare:      [...]

    Returns (rows, total_pages, current_page).
    """
    if payload is None:
        return [], 1, 1
    if isinstance(payload, list):
        return payload, 1, 1
    if isinstance(payload, dict) and "Data" in payload:
        return (
            payload.get("Data") or [],
            payload.get("TotalPages") or 1,
            payload.get("CurrentPage") or payload.get("Page") or 1,
        )
    return [payload], 1, 1

def get(self, path, params=None):
    """Single GET, shape-normalised to a list of rows."""

```

```

        rows, _, _ = self._unwrap(self.request("GET", path, params=params))
        return rows

def get_all(self, path, params=None, page_size=None):
    """
    GET every page. Yields rows.

    Metrc caps pageSize at 20 on many endpoints, so we clamp rather than
    letting the server reject the request.
    """
    params = dict(params or {})
    size = min(page_size or self.settings.default_page_size or MAX_PAGE_SIZE,
                MAX_PAGE_SIZE)
    params["pageSize"] = size
    page = 1

    while True:
        params["pageNumber"] = page
        rows, total_pages, _ = self._unwrap(
            self.request("GET", path, params=params)
        )
        for row in rows:
            yield row

        # A bare-array endpoint reports total_pages == 1 and is done.
        if page >= (total_pages or 1) or not rows:
            return
        page += 1

# ----- write API

def post(self, path, body, params=None, reference=None):
    return self.request("POST", path, params=params,
                        body=body, reference=reference)

def put(self, path, body, params=None, reference=None):
    return self.request("PUT", path, params=params,
                        body=body, reference=reference)

def delete(self, path, params=None, reference=None):
    return self.request("DELETE", path, params=params, reference=reference)

def post_chunked(self, path, objects, params=None, reference=None):
    """
    POST an arbitrarily long list, respecting the 10-object limit.

    Returns the concatenated "Ids" list, positionally aligned with
    `objects` – Metrc guarantees IDs come back in submission order.
    """
    ids = []

```

```

    for i in range(0, len(objects), MAX_OBJECTS_PER_REQUEST):
        chunk = objects[i:i + MAX_OBJECTS_PER_REQUEST]
        resp = self.post(path, chunk, params=params, reference=reference)
        if isinstance(resp, dict) and resp.get("Ids"):
            ids.extend(resp["Ids"])
        else:
            ids.extend([None] * len(chunk))
    return ids

def put_chunked(self, path, objects, params=None, reference=None):
    for i in range(0, len(objects), MAX_OBJECTS_PER_REQUEST):
        self.put(path, objects[i:i + MAX_OBJECTS_PER_REQUEST],
                params=params, reference=reference)

# ----- log

def _log(self, direction, method, url, body, status,
        response, duration_ms, error, reference):
    """
    Write a Metrc API Log row. Never logs headers – the API keys live
    in the Authorization header and must never reach the database.
    """
    try:
        doc = frappe.new_doc("Metrc API Log")
        doc.timestamp = now_datetime()
        doc.direction = direction
        doc.method = method
        doc.endpoint = url[:500]
        doc.license_number = self.license_number
        doc.request_body = json.dumps(body, indent=2)[:100000] if body else None
        doc.response_status = status
        doc.response_body = (
            json.dumps(response, indent=2)[:100000] if response is not None else None
        )
        doc.duration_ms = duration_ms
        doc.error = (error or "")[:1000] or None
        if reference:
            doc.reference_doctype, doc.reference_name = reference
        doc.flags.ignore_permissions = True
        doc.insert(ignore_permissions=True)
    except Exception:
        frappe.log_error(frappe.get_traceback(), "[metrc] failed to write API log")

def get_client(license_number):
    return MetrcClient(license_number)

```

6.1 Notes on the client

- **urlencode is not cosmetic.** It is what makes `%2B` correct. A hand-built query string will silently return wrong data for any timestamp with a positive UTC offset.

- `_unwrap` **is the single place** the two response shapes are reconciled. Everything downstream sees a list.
 - `post_chunked` **returns positionally-aligned IDs**, which is what lets you write Metrc IDs back onto the right ERPNext rows.
 - `dry_run` **short-circuits before transmission** but still logs the exact payload. This is your production rehearsal mechanism.
 - **Auth errors never retry.** Retrying a 401 four times just multiplies the audit noise.
-

7. The pull (inbound sync) engine

7.1 The cursor-window driver

```

# cannabis_management/metrc/pull/base.py
"""
Cursor-window sweep driver.

Metrc requires a bounded lastModified range on list endpoints and the spec is
explicit that sweeps must run oldest -> newest. LastModified only moves
forward; polling newest-first loses records that are modified mid-sweep.

Each (licence, endpoint) pair therefore keeps a persisted watermark and we
advance it one window at a time until we catch up to now.
"""

import frappe
from frappe.utils import add_to_date, get_datetime, now_datetime

from cannabis_management.metrc import config
from cannabis_management.metrc.client import get_client

# Never re-request the last few minutes as "complete" – Metrc's LastModified
# can lag slightly behind the write. Overlap absorbs that.
OVERLAP_MINUTES = 5

# How far back a brand-new cursor starts.
INITIAL_BACKFILL_DAYS = 90

def _iso(dt):
    """Metrc wants ISO 8601. urlencode handles the %2B escaping downstream."""
    return get_datetime(dt).strftime("%Y-%m-%dT%H:%M:%SZ")

def get_sync_state(license_number, endpoint_key):
    name = f"{license_number}:{endpoint_key}"
    if frappe.db.exists("Metrc Sync State", name):
        return frappe.get_doc("Metrc Sync State", name)

    doc = frappe.new_doc("Metrc Sync State")
    doc.name = name
    doc.license_number = license_number
    doc.endpoint_key = endpoint_key
    doc.cursor_last_modified = add_to_date(now_datetime(),
                                           days=-INITIAL_BACKFILL_DAYS)

    doc.flags.ignore_permissions = True
    doc.insert(ignore_permissions=True)
    return doc

def sweep(license_number, endpoint_key, path, handler,
          window_hours=None, extra_params=None):
    """

```

Advance the cursor for one (licence, endpoint) pair.

`handler(rows, license\_number)` upserts a page of rows and must be idempotent – windows overlap by design.

Returns the number of records processed.

```
"""
state = get_sync_state(license_number, endpoint_key)
settings = config.get_settings()
window_hours = window_hours or settings.window_hours or 24

client = get_client(license_number)
cursor = get_datetime(state.cursor_last_modified)
ceiling = now_datetime()
total = 0

state.last_run_start = now_datetime()

try:
    while cursor < ceiling:
        window_end = min(add_to_date(cursor, hours=window_hours), ceiling)

        params = dict(extra_params or {})
        params["lastModifiedStart"] = _iso(cursor)
        params["lastModifiedEnd"] = _iso(window_end)

        rows = list(client.get_all(path, params=params))
        if rows:
            handler(rows, license_number)
            total += len(rows)

        # Commit per window so a later failure does not replay work
        # that already succeeded.
        cursor = add_to_date(window_end, minutes=-OVERLAP_MINUTES)
        state.cursor_last_modified = cursor
        state.db_set("cursor_last_modified", cursor,
                    update_modified=False, commit=True)

    state.last_status = "Success"
    state.last_error = None
    state.consecutive_failures = 0

except Exception as e:
    state.last_status = "Failed"
    state.last_error = str(e)[:1000]
    state.consecutive_failures = (state.consecutive_failures or 0) + 1
    frappe.log_error(
        frappe.get_traceback(),
        f"[metrc] sweep failed {license_number}/{endpoint_key}",
    )
```

```
        raise

    finally:
        state.last_run_end = now_datetime()
        state.records_synced = (state.records_synced or 0) + total
        state.flags.ignore_permissions = True
        state.save(ignore_permissions=True)
        frappe.db.commit()

    return total
```

Why the cursor is committed per window: a 90-day backfill at 24-hour windows is 90 iterations. If iteration 80 fails, you do not want to redo 79 windows. Committing the watermark after each window makes the sweep resumable.

Why windows overlap by 5 minutes: Metrc's `LastModified` is set server-side and can lag microseconds-to-seconds behind the actual write. A hard boundary at exactly `window_end` can drop a record written in that gap. The overlap costs a few duplicate rows, which is free because handlers are idempotent.

7.2 Packages → Batch

```

# cannabis_management/metrc/pull/packages.py
"""Pull Metrc packages into ERPNext Batch + Metric Tag."""

import frappe
from frappe.utils import now_datetime

from cannabis_management.metrc import config
from cannabis_management.metrc.pull.base import sweep

def sync_packages(license_number):
    total = 0
    for key, path in (
        ("packages.active", "/packages/v2/active"),
        ("packages.inactive", "/packages/v2/inactive"),
        ("packages.onhold", "/packages/v2/onhold"),
    ):
        total += sweep(license_number, key, path, upsert_packages)
    return total

def upsert_packages(rows, license_number):
    """Idempotent upsert of a page of Metrc packages."""
    warehouse = config.warehouse_for_license(license_number)

    for pkg in rows:
        label = pkg.get("Label")
        if not label:
            continue
        try:
            _upsert_tag(pkg, label, warehouse)
            _upsert_batch(pkg, label, warehouse, license_number)
        except Exception:
            frappe.log_error(
                frappe.get_traceback(),
                f"[metrc] package upsert failed: {label}",
            )
    frappe.db.commit()

def _upsert_tag(pkg, label, warehouse):
    """Keep Metric Tag in step with the package's live state."""
    status = "Empty" if pkg.get("FinishedDate") else "Active"
    values = {
        "status": status,
        "current_qty": pkg.get("Quantity") or 0,
        "warehouse": warehouse,
        "last_transaction_type": "Metrc Sync",
        "last_updated": now_datetime(),
    }

```

```

if frappe.db.exists("Metric Tag", label):
    frappe.db.set_value("Metric Tag", label, values, update_modified=True)
    return

doc = frappe.new_doc("Metric Tag")
doc.tag_code = label
doc.muid = label[-6:]
doc.update(values)
doc.flags.ignore_permissions = True
doc.insert(ignore_permissions=True)

def _upsert_batch(pkg, label, warehouse, license_number):
    """
    Batch is the ERPNext side of a Metrc package.

    We only ever write the METRC_* mirror fields here. ERPNext's own
    quantity is owned by the stock ledger; divergence between the two is
    surfaced by reconcile.py, never silently patched.
    """
    batch_name = frappe.db.get_value("Batch", {"custom_metrc_tag": label}, "name")

    values = {
        "custom_metrc_package_id": pkg.get("Id"),
        "custom_metrc_quantity": pkg.get("Quantity") or 0,
        "custom_metrc_uom": pkg.get("UnitOfMeasureName"),
        "custom_metrc_status": _package_status(pkg),
        "custom_metrc_license_source": warehouse,
        "custom_metrc_last_synced": now_datetime(),
    }

    if batch_name:
        frappe.db.set_value("Batch", batch_name, values, update_modified=True)
        return

    item_code = _resolve_item(pkg)
    if not item_code:
        # A package for an item we do not carry. Recorded on the tag, but we
        # cannot make a Batch without a valid Item link.
        return

    doc = frappe.new_doc("Batch")
    doc.batch_id = label
    doc.item = item_code
    doc.custom_metrc_tag = label
    doc.custom_strain_name = _resolve_strain(pkg)
    doc.update(values)
    doc.flags.ignore_permissions = True
    doc.insert(ignore_permissions=True)

```

```

def _package_status(pkg):
    if pkg.get("FinishedDate"):
        return "Finished"
    if pkg.get("IsOnHold"):
        return "On Hold"
    if pkg.get("IsInTransit"):
        return "In Transit"
    return "Active"

def _resolve_item(pkg):
    """Map a Metrc item to an ERPNext Item by explicit mapping, then by name."""
    item = pkg.get("Item") or {}
    metrc_name = item.get("Name")
    if not metrc_name:
        return None

    return (
        frappe.db.get_value("Item", {"custom_metrc_item_name": metrc_name}, "name")
        or frappe.db.get_value("Item", {"item_name": metrc_name}, "name")
    )

def _resolve_strain(pkg):
    item = pkg.get("Item") or {}
    name = item.get("StrainName")
    if name and frappe.db.exists("Strain", name):
        return name
    return None

```

7.3 Available tags

```

# cannabis_management/metrc/pull/tags.py
"""
Pull the unused tag pool.

/tags/v2/*/available returns a BARE ARRAY, not a paginated envelope
(confirmed live). MetrcClient._unwrap handles this; do not add special cases.
"""

import frappe
from frappe.utils import now_datetime

from cannabis_management.metrc.client import get_client

def sync_available_tags(license_number):
    client = get_client(license_number)
    count = 0

    for path, tag_kind in (
        ("/tags/v2/package/available", "Package"),
        ("/tags/v2/plant/available", "Plant"),
    ):
        try:
            rows = client.get(path)
        except Exception:
            frappe.log_error(
                frappe.get_traceback(),
                f"[metrc] tag pull failed {license_number} {path}",
            )
            continue

        for tag in rows:
            label = tag.get("Label")
            if not label or frappe.db.exists("Metric Tag", label):
                continue
            doc = frappe.new_doc("Metric Tag")
            doc.tag_code = label
            doc.muid = label[-6:]
            doc.status = "Unused"
            doc.last_transaction_type = f"Metrc {tag_kind} Tag"
            doc.last_updated = now_datetime()
            doc.flags.ignore_permissions = True
            doc.insert(ignore_permissions=True)
            count += 1

    frappe.db.commit()
    return count

def claim_tag(license_number, kind="Package"):

```

```

"""
Reserve the lowest unused tag. Locks the row so two concurrent pushes
cannot claim the same physical label.
"""
rows = frappe.db.sql(
    """
    SELECT name FROM `tabMetric Tag`
    WHERE status = 'Unused' AND last_transaction_type = %s
    ORDER BY tag_code ASC LIMIT 1
    FOR UPDATE
    """,
    (f"Metric {kind} Tag",),
    as_dict=True,
)
if not rows:
    frappe.throw(f"No unused Metric {kind} tags available for {license_number}")

tag = rows[0].name
frappe.db.set_value("Metric Tag", tag, "status", "Active")
return tag

```

FOR UPDATE is essential. Two workers claiming the same tag means two Metric packages with one physical label — a compliance incident that cannot be undone from the API.

7.4 The orchestrator

```

# cannabis_management/metrc/pull/__init__.py
"""Scheduled pull orchestration."""

import frappe

from cannabis_management.metrc import config
from cannabis_management.metrc.pull import (
    harvests, items, labtests, packages, plantbatches, sales, strains,
    tags, transfers,
)

ALERT_AFTER_FAILURES = 3

def _run(fn, facility, label):
    try:
        return fn(facility.license_number) or 0
    except Exception:
        frappe.log_error(
            frappe.get_traceback(),
            f"[metrc] {label} failed for {facility.license_number}",
        )
        return 0

def sync_master_data():
    """Hourly: items, strains, tag pool. Small, cheap, changes rarely."""
    if not config.is_enabled():
        return
    for f in config.active_facilities():
        _run(items.sync_items, f, "items")
        _run(strains.sync_strains, f, "strains")
        _run(tags.sync_available_tags, f, "tags")

def sync_inventory():
    """Every 30 min: packages and transfers. The compliance-critical path."""
    if not config.is_enabled():
        return
    for f in config.active_facilities(feature="sync_packages"):
        _run(packages.sync_packages, f, "packages")
    for f in config.active_facilities(feature="sync_transfers"):
        _run(transfers.sync_transfers, f, "transfers")

def sync_operations():
    """Daily: sales, harvests, plant batches, lab tests."""
    if not config.is_enabled():
        return
    for f in config.active_facilities(feature="sync_sales"):

```

```

        _run(sales.sync_receipts, f, "sales")
    for f in config.active_facilities(feature="sync_harvests"):
        _run(harvests.sync_harvests, f, "harvests")
    for f in config.active_facilities(feature="sync_plants"):
        _run(plantbatches.sync_plantbatches, f, "plantbatches")
    for f in config.active_facilities(feature="sync_packages"):
        _run(labtests.sync_labtests, f, "labtests")

def alert_on_stalled_syncs():
    """Daily: email if any cursor has failed repeatedly."""
    stalled = frappe.get_all(
        "Metrc Sync State",
        filters={"consecutive_failures": [">=", ALERT_AFTER_FAILURES]},
        fields=["name", "consecutive_failures", "last_error"],
    )
    if not stalled:
        return

    to = frappe.db.get_single_value("Metrc Settings", "alert_email")
    if not to:
        return

    rows = "".join(
        f"<tr><td>{s.name}</td><td>{s.consecutive_failures}</td>"
        f"<td>{frappe.utils.escape_html(s.last_error or '')}</td></tr>"
        for s in stalled
    )
    frappe.sendmail(
        recipients=[to],
        subject=f"[METRC] {len(stalled)} sync cursor(s) stalled",
        message=(
            "<p>The following Metrc sync cursors have failed repeatedly:</p>"
            "<table border=1 cellpadding=4><tr><th>Cursor</th>"
            f"<th>Failures</th><th>Last error</th></tr>{rows}</table>"
        ),
    )

```

8. The push (outbound) engine

8.1 Why an outbox, not a direct call

Calling Metrc from a `on_submit` hook couples your ERP to a third-party state system's uptime. A Metrc 500 during Sales Invoice submission would either roll back the invoice or leave you submitted-but-unreported. Neither is acceptable.

The outbox pattern decouples them:

1. `on_submit` writes a `Metrc Outbox` row in the **same database transaction** as the document. Either both commit or neither does.
2. A background worker drains the outbox with retries and backoff.
3. Failures are visible, replayable, and never block operations.

8.2 Idempotency

Every outbox row carries a unique `idempotency_key` derived from the source document and operation:

```
sha256(f"{operation}:{reference_doctype}:{reference_name}:{discriminator}")
```

The unique index makes double-enqueue a no-op. Combined with the "check before create" guard in each push handler, a retry after an ambiguous timeout cannot create a duplicate Metrc record.

Metrc has no server-side idempotency keys. If a POST times out after Metrc committed it, you cannot know. So every push handler must **verify before creating**: query Metrc for the record (by external ID, tag, or receipt number) before submitting. `GET /sales/v2/receipts/external/{externalNumber}` exists precisely for this.

8.3 Implementation

```

# cannabis_management/metrc/push/outbox.py
"""
Transactional outbox for Metrc writes.

Documents enqueue; a scheduled worker drains. This keeps ERPNext operations
independent of Metrc availability and makes every write replayable.
"""

import hashlib
import json

import frappe
from frappe.utils import add_to_date, now_datetime

from cannabis_management.metrc import config
from cannabis_management.metrc.client import get_client
from cannabis_management.metrc.exceptions import (
    MetrcAuthError,
    MetrcPayloadTooLargeError,
    MetrcValidationError,
)

BATCH_SIZE = 50
MAX_ATTEMPTS = 6

# Errors that will never succeed on retry – park immediately for human review.
TERMINAL = (MetrcValidationError, MetrcAuthError, MetrcPayloadTooLargeError)

def make_key(operation, ref_doctype, ref_name, discriminator=""):
    raw = f"{operation}:{ref_doctype}:{ref_name}:{discriminator}"
    return hashlib.sha256(raw.encode()).hexdigest()[:40]

def enqueue(operation, license_number, payload,
            reference_doctype=None, reference_name=None, discriminator=""):
    """
    Queue a Metrc write. Safe to call twice – the unique idempotency key
    makes the second call a no-op.

    Call this from document hooks; it participates in the caller's
    transaction, so the outbox row and the document commit together.
    """
    key = make_key(operation, reference_doctype, reference_name, discriminator)
    if frappe.db.exists("Metrc Outbox", {"idempotency_key": key}):
        return None

    doc = frappe.new_doc("Metrc Outbox")
    doc.status = "Queued"
    doc.operation = operation

```

```

doc.license_number = license_number
doc.payload = json.dumps(payload, indent=2, default=str)
doc.idempotency_key = key
doc.reference_doctype = reference_doctype
doc.reference_name = reference_name
doc.next_attempt_at = now_datetime()
doc.flags.ignore_permissions = True
doc.insert(ignore_permissions=True)
return doc.name

def process_outbox():
    """Scheduled worker. Drains queued and due-for-retry rows."""
    if not config.is_enabled():
        return
    if not frappe.db.get_single_value("Metrc Settings", "push_enabled"):
        return

    rows = frappe.get_all(
        "Metrc Outbox",
        filters={
            "status": ["in", ["Queued", "Failed"]],
            "next_attempt_at": ["<=", now_datetime()],
        },
        fields=["name"],
        order_by="creation asc",
        limit=BATCH_SIZE,
    )
    for row in rows:
        _process_one(row.name)

def _process_one(name):
    from cannabis_management.metrc.push import HANDLERS

    doc = frappe.get_doc("Metrc Outbox", name)
    doc.db_set("status", "In Progress", update_modified=False, commit=True)

    handler = HANDLERS.get(doc.operation)
    if not handler:
        _park(doc, f"No handler registered for operation '{doc.operation}'")
        return

    try:
        client = get_client(doc.license_number)
        payload = json.loads(doc.payload)
        result = handler(client, payload, doc)

        doc.db_set({
            "status": "Success",

```

```

        "attempts": (doc.attempts or 0) + 1,
        "response": json.dumps(result, indent=2, default=str),
        "metrc_id": _extract_id(result),
        "last_error": None,
    }, update_modified=False, commit=True)

except TERMINAL as e:
    _park(doc, str(e))

except Exception as e:
    attempts = (doc.attempts or 0) + 1
    if attempts >= MAX_ATTEMPTS:
        _park(doc, f"Gave up after {attempts} attempts: {e}")
        return
    # 2, 4, 8, 16, 32 minutes
    doc.db_set({
        "status": "Failed",
        "attempts": attempts,
        "last_error": str(e)[:1000],
        "next_attempt_at": add_to_date(now_datetime(),
                                       minutes=2 ** attempts),
    }, update_modified=False, commit=True)

def _park(doc, error):
    doc.db_set({
        "status": "Parked",
        "attempts": (doc.attempts or 0) + 1,
        "last_error": error[:1000],
    }, update_modified=False, commit=True)
    frappe.log_error(
        f"Outbox {doc.name} ({doc.operation}) parked:\n{error}",
        "[metrc] outbox parked",
    )

def _extract_id(result):
    if isinstance(result, dict):
        ids = result.get("Ids")
        if ids:
            return str(ids[0])
        if result.get("Id"):
            return str(result["Id"])
    if isinstance(result, list) and result:
        return str(result[0])
    return None

```

```
# cannabis_management/metrc/push/__init__.py
"""Operation -> handler registry for the outbox worker."""

from cannabis_management.metrc.push import packages, processing, sales, transfers

HANDLERS = {
    "packages.create": packages.create_package,
    "packages.adjust": packages.adjust_package,
    "packages.finish": packages.finish_package,
    "packages.change_item": packages.change_item,
    "packages.change_location": packages.change_location,
    "sales.receipt.create": sales.create_receipt,
    "sales.receipt.update": sales.update_receipt,
    "transfers.external_incoming.create": transfers.create_incoming,
    "transfers.template.create": transfers.create_template,
    "processing.start": processing.start_job,
    "processing.createpackages": processing.create_packages,
    "processing.finish": processing.finish_job,
}
```

9. Flow A — Distribution & sales

This is the recommended **first production flow**: it exercises auth, pagination, cursors, rate limits and writes, and it maps onto documents your team already uses daily.

9.1 Sales Invoice → Metrc Sales Receipt

The `SalesDateTime` timezone rule from §2.3 is the critical detail here.

```

# cannabis_management/metrc/push/sales.py
"""
Sales Invoice -> Metrc sales receipt.

CRITICAL: SalesDateTime must be the facility's LOCAL wall-clock time with no
timezone suffix. Metrc interprets it as facility-local. Sending UTC shifts
every receipt by the UTC offset (8 hours for California) and will push
transactions into the wrong reporting day.
"""

from datetime import datetime
from zoneinfo import ZoneInfo

import frappe
from frappe.utils import get_datetime

from cannabis_management.metrc import config
from cannabis_management.metrc.push.outbox import enqueue

DEFAULT_TZ = "America/Los_Angeles"

def facility_local_naive(dt, license_number):
    """
    Convert a Frappe datetime (stored in the site timezone) to the facility's
    local wall clock, then drop the tzinfo – Metrc wants it naive.
    """
    tz_name = DEFAULT_TZ
    try:
        tz_name = config.get_facility(license_number).facility_timezone or DEFAULT_TZ
    except Exception:
        pass

    dt = get_datetime(dt)
    site_tz = ZoneInfo(frappe.utils.get_system_timezone())
    if dt.tzinfo is None:
        dt = dt.replace(tzinfo=site_tz)

    local = dt.astimezone(ZoneInfo(tz_name))
    return local.replace(tzinfo=None).strftime("%Y-%m-%dT%H:%M:%S.000")

def build_receipt(invoice, license_number):
    """Build the Metrc receipt object for a submitted Sales Invoice."""
    transactions = []

    for item in invoice.items:
        tag = _tag_for_item_row(item)
        if not tag:
            frappe.throw(

```

```

        f"Sales Invoice {invoice.name} row {item.idx} ({item.item_code}) "
        "has no METRC package tag. Set a Batch with custom_metrc_tag."
    )
transactions.append({
    "PackageLabel": tag,
    "Quantity": float(item.qty),
    "UnitOfMeasure": _metrc_uom(item.uom),
    "TotalAmount": float(item.amount),
    "UnitThcPercent": None,
    "UnitThcContent": None,
    "UnitThcContentUnitOfMeasure": None,
    "UnitWeight": None,
    "UnitWeightUnitOfMeasure": None,
    "InvoiceNumber": invoice.name,
    "Price": float(item.rate),
    "ExciseTax": None,
    "CityTax": None,
    "CountyTax": None,
    "MunicipalTax": None,
    "DiscountAmount": float(item.discount_amount or 0),
    "SubTotal": float(item.net_amount),
    "SalesTax": None,
    "QrCodes": None,
})

return {
    "SalesDateTime": facility_local_naive(
        invoice.posting_date and
        f"{invoice.posting_date} {invoice.posting_time or '00:00:00'}",
        license_number,
    ),
    "ExternalReceiptNumber": invoice.name, # our idempotency handle
    "SalesCustomerType": _customer_type(invoice),
    "PatientLicenseNumber": None,
    "CaregiverLicenseNumber": None,
    "IdentificationMethod": None,
    "PatientRegistrationLocationId": None,
    "Transactions": transactions,
}

def _tag_for_item_row(item):
    if item.get("batch_no"):
        tag = frappe.db.get_value("Batch", item.batch_no, "custom_metrc_tag")
        if tag:
            return tag
    return None

def _metrc_uom(erpnext_uom):

```

```

from cannabis_management.metrc.mapping import to_metrc_uom
return to_metrc_uom(erpnext_uom)

def _customer_type(invoice):
    # Extend with your Customer classification. Consumer is the default for
    # adult-use retail; Patient/Caregiver require licence numbers.
    return "Consumer"

# ----- hooks

def on_sales_invoice_submit(doc, method=None):
    """Enqueue a Metrc receipt. Runs inside the submit transaction."""
    if not config.is_enabled():
        return
    if doc.get("custom_metrc_sync_status") == "Not Required":
        return

    license_number = config.license_for_warehouse(
        doc.set_warehouse or (doc.items[0].warehouse if doc.items else None)
    )
    if not license_number:
        return

    try:
        facility = config.get_facility(license_number)
    except Exception:
        return
    if not facility.sync_sales:
        return

    payload = build_receipt(doc, license_number)
    enqueue(
        operation="sales.receipt.create",
        license_number=license_number,
        payload=payload,
        reference_doctype="Sales Invoice",
        reference_name=doc.name,
    )
    doc.db_set("custom_metrc_sync_status", "Queued",
              update_modified=False)

# ----- handlers

def create_receipt(client, payload, outbox_doc):
    """
    Outbox handler.

```

Metrc has no idempotency keys, so we check whether the receipt already exists by our ExternalReceiptNumber before creating. This makes a retry after an ambiguous timeout safe.

```
"""
external = payload.get("ExternalReceiptNumber")
if external:
    try:
        existing = client.get(f"/sales/v2/receipts/external/{external}")
        if existing:
            row = existing[0] if isinstance(existing, list) else existing
            _write_back(outbox_doc, row.get("Id"))
            return {"Ids": [row.get("Id")], "AlreadyExisted": True}
    except Exception:
        # 404 / not-found is the normal path – proceed to create.
        pass

result = client.post("/sales/v2/receipts", [payload],
                    reference=(outbox_doc.reference_doctype,
                              outbox_doc.reference_name))

ids = (result or {}).get("Ids") or []
if ids:
    _write_back(outbox_doc, ids[0])
return result
```

```
def update_receipt(client, payload, outbox_doc):
    return client.put("/sales/v2/receipts", [payload],
                    reference=(outbox_doc.reference_doctype,
                              outbox_doc.reference_name))
```

```
def _write_back(outbox_doc, metrc_id):
    if outbox_doc.reference_doctype != "Sales Invoice":
        return
    frappe.db.set_value(
        "Sales Invoice", outbox_doc.reference_name,
        {
            "custom_metrc_receipt_id": str(metrc_id),
            "custom_metrc_sync_status": "Synced",
        },
        update_modified=False,
    )
```

9.2 Delivery Note → outgoing transfer

Outgoing transfers are the most complex payload in the API. The structure is three levels deep:

```

Transfer
└─ Destinations[]          (one per recipient licence)
    └─ Transporters[]      (one per leg)
        └─ TransporterDetails[] (driver/vehicle per layover leg)
            └─ Packages[]   (the tags being moved)

```

Your `si_manifest_cdt` child table already holds manifest data — map it to `Destinations[0]`.

Note the API asymmetry: `POST /transfers/v2/external/incoming` creates incoming shipment plans, and outgoing transfers are created from **templates** (`POST /transfers/v2/templates/outgoing`) or through the Metrc UI. There is no plain "create outgoing transfer" endpoint. Build outgoing transfers as templates and have operations promote them, or record them as external incoming from the receiving side.

9.3 Incoming transfers → Purchase Receipt

Pull-side. The chain requires three calls per transfer, and the spec explicitly warns about rate limiting here:

```

GET /transfers/v2/incoming          → transfer IDs
GET /transfers/v2/{id}/deliveries    → delivery IDs   (1 call per transfer)
GET /transfers/v2/deliveries/{id}/packages → package tags (1 call per delivery)

```

With N incoming transfers you make 1 + N + M calls. **Cache aggressively and only walk the chain for transfers whose `LastModified` advanced.** Store the transfer ID on the Purchase Receipt so you never re-walk a completed one.

10. Flow B — Manufacturing / processing

Maps to your `Conversion Entry` , `Manufacture Stock Entry` and `Work Order` doctypes.

10.1 The Metrc processing lifecycle

<code>POST /processing/v2/start</code>	→ job created, input packages consumed
<code>POST /processing/v2/adjust</code>	→ correct inputs/UOMs mid-job
<code>POST /processing/v2/createpackages</code>	→ output packages created (can finish the job)
<code>PUT /processing/v2/finish</code>	→ close the job

`POST /processing/v2/createpackages` accepts `FinishProcessingJob: true` plus waste quantities, which closes the job and records waste in one call. Use that rather than a separate finish call — it is one fewer request against your rate limit and it is atomic.

10.2 Mapping

ERPNext event	Metrc call
Work Order / Conversion Entry submitted	<code>POST /processing/v2/start</code> with input Batch tags
Manufacture Stock Entry (Manufacturing type) submitted	<code>POST /processing/v2/createpackages</code> with output tags, <code>FinishProcessingJob: true</code>
Stock Entry — Material Issue for waste	waste fields on <code>createpackages</code>

```
# cannabis_management/metrc/push/processing.py (sketch)

def build_start_job(work_order, license_number):
    """Conversion Entry / Work Order -> POST /processing/v2/start"""
    packages = []
    for row in work_order.required_items:
        tag = frappe.db.get_value("Batch", row.batch_no, "custom_metrc_tag")
        if not tag:
            frappe.throw(f"Input row {row.idx} has no METRC tag")
        packages.append({
            "Label": tag,
            "Quantity": float(row.required_qty),
            "UnitOfMeasure": to_metrc_uom(row.stock_uom),
        })

    return {
        "JobName": work_order.name,
        "JobType": work_order.get("custom_metrc_job_type") or "Infusing",
        "CountUnitOfMeasure": "Each",
        "VolumeUnitOfMeasure": "Fluid Ounces",
        "WeightUnitOfMeasure": "Grams",
        "Packages": packages,
        "StartDate": f"{work_order.planned_start_date}T00:00:00Z",
    }

def build_create_packages(mse, license_number):
    """Manufacture Stock Entry -> POST /processing/v2/createpackages"""
    out = []
    for row in mse.manufacture_finished_goods:
        out.append({
            "JobName": mse.get("custom_metrc_job_name"),
            "Tag": claim_tag(license_number, "Package"),
            "Location": None,
            "Sublocation": None,
            "Item": frappe.db.get_value("Item", row.item_code,
                                       "custom_metrc_item_name"),
            "Quantity": float(row.qty),
            "UnitOfMeasure": to_metrc_uom(row.uom),
            "IsFinishedGood": True,
            "PatientLicenseNumber": None,
            "Note": None,
            "ProductionBatchNumber": mse.name,
            "FinishProcessingJob": True,
            "FinishDate": str(mse.date),
            "WasteWeightQuantity": float(mse.get("custom_waste_qty") or 0) or None,
            "WasteWeightUnitOfMeasureName": "Grams",
            "FinishNote": None,
            "PackageDate": str(mse.date),
            "ExpirationDate": None,
        })
```

```
        "SellByDate": None,  
        "UseByDate": None,  
    })  
    return out
```

JobType must exist in Metrc. Pull the valid list from [GET /processing/v2/jobtypes/active](#) and store it as a Select's options, refreshed by the master-data sync. Do not hardcode.

11. Flow C — Cultivation

Maps to `Cloning Batch`, `Farm Production Batch`, `Production Batch`, `Farm Daily Log`.

11.1 Lifecycle

POST <code>/plantbatches/v2/plantings</code>	Cloning Batch → immature plant batch
POST <code>/plantbatches/v2/growthphase</code>	growth phase change → veg plants (tagged)
PUT <code>/plants/v2/growthphase</code>	veg → flowering
PUT <code>/plants/v2/location</code>	plant moves
POST <code>/plants/v2/additives</code>	nutrient application (Farm Daily Log)
PUT <code>/plants/v2/harvest</code>	Production Batch → harvest, wet weight
POST <code>/harvests/v2/waste</code>	waste removal
POST <code>/harvests/v2/packages</code>	harvest → tagged packages
PUT <code>/harvests/v2/finish</code>	close the harvest

11.2 Notes specific to cultivation

- **Plant batches are referenced by name**, not just ID. Use `Cloning Batch.name` as `PlantBatchName` and the mapping is free.
 - **PUT `/plants/v2/harvest` auto-generates the harvest name if you pass `HarvestName: null`**. Prefer sending your own (`Production Batch.name`) so the two systems share a key.
 - **Wet weight at harvest is a compliance figure**. It must be captured at the scale, not derived. Route it from `Farm Daily Log`.
 - **Waste must be reported separately** via `POST /harvests/v2/waste` with a reason from `GET /harvests/v2/waste/types`. Do not fold waste into an adjustment.
 - **GET `/plants/v2/growthphases` and `/plants/v2/waste/reasons`** are enumerations — pull them into Select options during master-data sync.
-

12. Reconciliation & variance reporting

This is the payoff, and it delivers value before any write goes live.

```

# cannabis_management/metrc/reconcile.py
"""
Compare Metrc package quantities against ERPNext stock and report variance.

Never auto-corrects. A quantity difference is either a data-entry error or a
genuine physical discrepancy; both need a human and both may need a Stock
Reconciliation with custom_metrc_correction_made set.
"""

import frappe
from frappe.utils import flt, now_datetime

TOLERANCE = 0.01

def find_variances(license_number=None):
    filters = {"custom_metrc_tag": ["is", "set"]}
    if license_number:
        wh = frappe.db.get_value(
            "Warehouse", {"custom_metrc_license_number": license_number}, "name"
        )
        if wh:
            filters["custom_metrc_license_source"] = wh

    batches = frappe.get_all(
        "Batch", filters=filters,
        fields=["name", "item", "custom_metrc_tag", "custom_metrc_quantity",
            "custom_metrc_uom", "custom_metrc_status",
            "custom_metrc_last_synced"],
    )

    variances = []
    for b in batches:
        erp_qty = flt(frappe.db.get_value(
            "Stock Ledger Entry",
            {"batch_no": b.name, "is_cancelled": 0},
            "sum(actual_qty)",
        ) or 0)
        metrc_qty = flt(b.custom_metrc_quantity)

        if abs(erp_qty - metrc_qty) > TOLERANCE:
            variances.append({
                "batch": b.name,
                "item": b.item,
                "metrc_tag": b.custom_metrc_tag,
                "metrc_qty": metrc_qty,
                "erpnext_qty": erp_qty,
                "difference": erp_qty - metrc_qty,
                "metrc_status": b.custom_metrc_status,
                "last_synced": b.custom_metrc_last_synced,
            })

```

```

    })

    return sorted(variances, key=lambda v: abs(v["difference"]), reverse=True)

def find_orphans():
    """Tags active in Metrc with no corresponding ERPNext Batch."""
    return frappe.db.sql("""
        SELECT mt.tag_code, mt.item_code, mt.current_qty, mt.warehouse
        FROM `tabMetric Tag` mt
        LEFT JOIN `tabBatch` b ON b.custom_metrc_tag = mt.tag_code
        WHERE mt.status = 'Active' AND b.name IS NULL
    """, as_dict=True)

def send_daily_variance_report():
    variances = find_variances()
    orphans = find_orphans()
    if not variances and not orphans:
        return

    to = frappe.db.get_single_value("Metrc Settings", "alert_email")
    if not to:
        return

    rows = "".join(
        f"<tr><td>{v['metrc_tag']}

```

Pair this with a Query Report ([metrc_variance_report](#)) so operations can work the list interactively.

13. Hooks wiring

Append to the existing `scheduler_events` in `hooks.py`:

```
scheduler_events = {
    "cron": {
        # ... existing entries unchanged ...

        # METRC – master data (items, strains, tag pool): hourly
        "15 * * * *": [
            "cannabis_management.metrc.pull.sync_master_data",
        ],
        # METRC – inventory (packages, transfers): every 30 minutes
        "*/30 * * * *": [
            "cannabis_management.metrc.pull.sync_inventory",
        ],
        # METRC – outbox worker: every 5 minutes
        "*/5 * * * *": [
            "cannabis_management.metrc.push.outbox.process_outbox",
        ],
        # METRC – operations (sales, harvests, plants, lab tests): 02:00 UTC
        "0 2 * * *": [
            "cannabis_management.metrc.pull.sync_operations",
        ],
        # METRC – variance + stalled-cursor alerts: 06:00 UTC
        "0 6 * * *": [
            "cannabis_management.metrc.reconcile.send_daily_variance_report",
            "cannabis_management.metrc.pull.alert_on_stalled_syncs",
        ],
        # METRC – log pruning: Sunday 03:00 UTC
        "0 3 * * 0": [
            "cannabis_management.metrc.maintenance.prune_logs",
        ],
    },
}
```

Document events:

```

doc_events = {
  # ... existing entries unchanged ...

  "Sales Invoice": {
    "on_submit": "cannabis_management.metroc.push.sales.on_sales_invoice_submit",
  },
  "Delivery Note": {
    "on_submit": "cannabis_management.metroc.push.transfers.on_delivery_note_submit",
  },
  "Stock Reconciliation": {
    "on_submit": "cannabis_management.metroc.push.packages.on_stock_reco_submit",
  },
  "Work Order": {
    "on_submit": "cannabis_management.metroc.push.processing.on_work_order_submit",
  },
  "Manufacture Stock Entry": {
    "on_submit": "cannabis_management.metroc.push.processing.on_mse_submit",
  },
}

```

Cadence rationale. Inventory at 30 minutes balances rate limit against compliance freshness — Metrc does not require real-time reporting, but a same-day discrepancy is far cheaper to resolve than a week-old one. The outbox at 5 minutes keeps writes prompt without hammering the API. Master data hourly is generous; items and strains change rarely.

14. Testing against the sandbox

14.1 Sandbox-only helper endpoints

These exist only on sandbox and return **403 on production**:

Endpoint	Purpose
POST /sandbox/v2/integrator/setup	Create/look up a sandbox user key. Without <code>userKey</code> it queues creation and emails the key (201/202/200). With <code>userKey</code> it returns the key as a plain string.
POST /sandbox/v2/facility/tags	Mint tags. Body <code>{"TagType": "Cannabis Package", "Count": 100}</code> . Max 1,000. Created, shipped and received synchronously — immediately usable.
POST /sandbox/v2/packages/create	Create opening-balance packages. Body <code>{"Count": 10, "FilterBy": "Category", "FilterValue": "Buds"}</code> . <code>FilterBy</code> ∈ <code>Name / Category / UnitOfMeasure</code> ; <code>FilterValue</code> required when <code>FilterBy</code> is set. Default 10, max 100.
GET /sandbox/v2/tagtypes	Valid tag types for this facility.

This is your test-fixture toolkit — you can build any scenario without a Metrc rep.

14.2 Seeding a test facility

```
INT='<integrator-key>'; U='<user-key>'
B='https://sandbox-api-ca.metrc.com'; L='C12-1000001-LIC'

# 1. Mint package tags
curl -s -u "$INT:$U" -H 'Content-Type: application/json' \
  -X POST "$B/sandbox/v2/facility/tags?licenseNumber=$L" \
  -d '{"TagType": "Cannabis Package", "Count": 100}'

# 2. Create opening-balance packages
curl -s -u "$INT:$U" -H 'Content-Type: application/json' \
  -X POST "$B/sandbox/v2/packages/create?licenseNumber=$L" \
  -d '{"Count": 25}'

# 3. Verify
curl -s -u "$INT:$U" \
  "$B/packages/v2/active?licenseNumber=$L&pageSize=20&pageNumber=1"
```

14.3 Test matrix

#	Scenario	Expected
1	Wrong keys	<code>MetrcAuthError</code> , zero retries , one log row
2	11 objects in a POST array	<code>MetrcPayloadTooLargeError</code> raised client-side , no HTTP call
3	25 objects via <code>post_chunked</code>	3 requests (10/10/5), 25 IDs in order
4	<code>lastModifiedStart</code> with <code>+02:00</code> offset	URL contains <code>%2B</code> , results correct
5	Bare-array endpoint (<code>/tags/v2/package/available</code>)	<code>_unwrap</code> returns list, <code>total_pages == 1</code>
6	Paginated endpoint, 45 records, pageSize 20	3 requests, 45 rows yielded
7	<code>pageSize=100</code> requested	Clamped to 20, no rejection
8	Cursor sweep, 90-day backfill, 24h windows	90 windows, watermark committed each time
9	Sweep fails at window 40	Watermark at window 39; rerun resumes there
10	Same package pulled twice	One Batch, one Metric Tag (idempotent)
11	<code>enqueue()</code> called twice for one invoice	One outbox row (unique key)
12	Outbox handler raises <code>MetrcValidationError</code>	Status <code>Parked</code> immediately, no retry
13	Outbox handler raises <code>MetrcServerError</code>	Status <code>Failed</code> , <code>next_attempt_at</code> +2 min
14	6 consecutive failures	Status <code>Parked</code> , error logged
15	Receipt retried after simulated timeout	External-number lookup finds it, no duplicate
16	<code>dry_run = 1</code>	Payload logged, <code>response_status = 0</code> , nothing transmitted
17	Two workers claim a tag concurrently	<code>FOR UPDATE</code> serialises; distinct tags
18	<code>SalesDateTime</code> for a 22:00 PDT invoice	<code>...T22:00:00.000</code> , not the UTC 05:00 next day
19	Metrc qty 100g, ERPNext 98g	Variance report shows <code>-2.0000</code>
20	Metrc tag active, no Batch	Appears in <code>find_orphans()</code>

Test 18 is the one that most often ships broken. Assert it explicitly.

14.4 Manual smoke test

```
bench --site stage.alltechvirtual.com console
```

```
from cannabis_management.metroc.client import get_client
c = get_client("C12-1000001-LIC")

len(list(c.get_all("/packages/v2/active")))
c.get("/unitsofmeasure/v2/active")          # bare array path
c.get("/tags/v2/package/available")[:3]     # bare array path

from cannabis_management.metroc.pull.packages import sync_packages
sync_packages("C12-1000001-LIC")

from cannabis_management.metroc.reconcile import find_variances
find_variances()[:10]
```

15. Production go-live checklist

Phase 1 — Read-only (weeks 1-2)

- [] Create the six doctypes and the custom fields
- [] Configure Metrc Settings: Sandbox, integrator key, one facility
- [] Map every ERPNext UOM used on tracked items to one of the 11 Metrc UOMs
- [] Map Warehouse ↔ licence for every facility
- [] Run the client smoke tests above
- [] Enable `sync_master_data` and `sync_inventory`
- [] Let cursors backfill 90 days; confirm they reach `Success`
- [] Review the variance report — expect noise on the first run, and work it down

Phase 2 — Push in dry-run (weeks 3-4)

- [] `push_enabled = 1`, `dry_run = 1`
- [] Wire `Sales Invoice.on_submit`
- [] Submit real invoices; inspect the logged payloads
- [] Verify `SalesDateTime` is facility-local (test 18)
- [] Verify every line resolves to a package tag
- [] Confirm chunking on a >10-line invoice

Phase 3 — Sandbox writes (weeks 5-6)

- [] `dry_run = 0`, still pointed at Sandbox
- [] Push receipts; verify via `GET /sales/v2/receipts/active`
- [] Test the retry-safety path (test 15)
- [] Exercise the full test matrix
- [] Run for a week; outbox should be empty of `Parked` rows

Phase 4 — Metrc validation

- [] Request production access from your Metrc representative
- [] Demo the integration in sandbox
- [] Confirm your rate-limit tier and whether webhooks are included
- [] Receive production integrator key approval

Phase 5 — Production (staged)

- [] **Rotate every key** used during development (\$16)
- [] `environment = Production`, `push_enabled = 0` — **read-only first**
- [] Backfill and reconcile against real Metrc data; resolve variances
- [] `push_enabled = 1`, `dry_run = 1` for one week on production data
- [] `dry_run = 0` for **one facility only**
- [] Monitor daily for two weeks
- [] Roll out to remaining facilities one at a time

Do not skip the read-only-on-production step. It is where you will find the differences between sandbox seed data and your actual operation.

16. Security

16.1 Rotate the keys used during development

The integrator key and the CA sandbox user key were pasted into a chat transcript and appear in shell history on this machine. Metrc's documentation is explicit that the integrator key must not be shared. Before production:

1. Regenerate the integrator key in Metrc Connect.
2. Regenerate the user key in the Metrc UI (the **Generate** button on the API Keys page).
3. Clear shell history: `history -c && rm -f ~/.bash_history` (note that `~/.bash_history` on this host currently contains prior sessions).
4. Store the new keys only in the `Password`-type fields on Metrc Settings — Frappe encrypts these with the site `encryption_key`.

16.2 Never commit keys

- Password fieldtypes are encrypted at rest and never appear in `bench export-fixtures`.
- **Exclude Metrc Settings from any fixtures export** — a Single doctype in fixtures would round-trip the encrypted values into git.
- Never log headers. The client above logs URL, body and response only, by design.

16.3 Permissions

- Metrc Settings: System Manager only.
- Metrc API Log / Metrc Outbox: read for Compliance Officer, write for System Manager.
- Metric Tag: read for Stock User, write only via the sync — a hand-edited tag is a compliance risk.

16.4 Audit retention

`log_retention_days` defaults to 120. Check California's record-retention requirement for your licence types before pruning; if it exceeds 120 days, raise the setting rather than archiving elsewhere.

16.5 Log pruning

```
# cannabis_management/metrc/maintenance.py

import frappe
from frappe.utils import add_days, now_datetime

def prune_logs():
    days = frappe.db.get_single_value("Metrc Settings", "log_retention_days") or 120
    cutoff = add_days(now_datetime(), -days)

    frappe.db.delete("Metrc API Log", {"timestamp": ["<", cutoff]})
    # Keep Parked rows regardless of age – they are unresolved compliance work.
    frappe.db.delete("Metrc Outbox", {
        "status": "Success",
        "modified": ["<", cutoff],
    })
    frappe.db.commit()
```


Appendix A — Endpoint reference

Module sizes are counts of documented operations across v1 and v2 (including examples).

Module	Ops	Key v2 endpoints
Sales	87	receipts/active, receipts/inactive, receipts/{id}, receipts/external/{n}, POST/PUT receipts, receipts/finalize, deliveries/*, deliveries/retailer/*
Transfers	78	incoming, outgoing, rejected, hub, {id}/deliveries, deliveries/{id}/packages, templates/outgoing, manifest/{id}/pdf, external/incoming
Plants	60	vegetative, flowering, onhold, inactive, mother, PUT growthphase / location / harvest / tag / strain / merge / split, POST plantings / waste / additives
Packages	58	active, inactive, onhold, intransit, transferred, labsamples, {label}, POST /, testing, PUT adjust / finish / unfinish / item / location / note / remediate / externalid
Plant Batches	39	active, inactive, types, POST plantings / packages / split / growthphase / waste, PUT name / tag / strain / location
Processing	38	active, inactive, jobtypes/*, POST start / adjust / createpackages, PUT finish / unfinish
Items	33	active, inactive, categories, brands, POST /, PUT /, photo, file
Harvests	30	active, onhold, inactive, waste, waste/types, POST packages / packages/testing / waste, PUT finish / rename / location
Patients	18	active, {id}, statuses/{lic}
Lab Tests	17	states, types, results, batches, POST record, PUT results/release, labtestdocument
Locations	17	active, inactive, types, POST/PUT/DELETE
Strains	15	active, inactive, POST/PUT/DELETE
Transporters	14	drivers, vehicles (full CRUD)
Patient Check-ins	12	full CRUD
Retail ID	9	allotment, associate, generate, merge, receive/{label}
Sublocations	8	active, inactive, CRUD
Additives Templates	6	active, inactive, POST/PUT
Caregivers	5	status/{lic}
Sandbox	5	integrator/setup, packages/create, facility/tags, tagtypes
Employees	3	/ (note: permissions returns 404 in CA)
Tags	3	plant/available, package/available, staged
Units of Measure	3	active, inactive
Webhooks	3	PUT/DELETE (contract-gated)
Facilities	2	/
	1	/

Module	Ops	Key v2 endpoints
Waste Methods		

Enumeration endpoints to cache

Pull these during master-data sync and use them as Select options. Never hardcode.

/unitsofmeasure/v2/active	/items/v2/categories
/packages/v2/types	/packages/v2/adjust/reasons
/plants/v2/growthphases	/plants/v2/waste/reasons
/plants/v2/waste/methods/all	/plantbatches/v2/types
/plantbatches/v2/waste/reasons	/harvests/v2/waste/types
/processing/v2/jobtypes/active	/processing/v2/jobtypes/categories
/transfers/v2/types	/sales/v2/customertypes
/sales/v2/paymenttypes	/labtests/v2/types
/locations/v2/types	/wastemethods/v2/

Appendix B — Payload schemas

All verbatim from the CA spec. All arrays capped at 10 objects.

POST /packages/v2/?licenseNumber=

```
[{
  "Tag": "ABCDEF012345670000020201",
  "Location": null, "Sublocation": null,
  "Item": "Buds",
  "Quantity": 16.0, "UnitOfMeasure": "Ounces",
  "PatientLicenseNumber": "X00001",
  "Note": "This is a note.",
  "IsProductionBatch": false, "ProductionBatchNumber": null,
  "IsDonation": false, "IsTradeSample": false, "IsFinishedGood": null,
  "ProductRequiresRemediation": false, "UseSameItem": false,
  "ActualDate": "2015-12-15",
  "ExpirationDate": null, "SellByDate": null, "UseByDate": null,
  "Ingredients": [
    { "Package": "ABCDEF012345670000010041", "Quantity": 8.0, "UnitOfMeasure": "Ounces" }
  ],
  "RequiredLabTestBatches": null,
  "ProcessingJobTypeId": null, "LabTestStageId": null
}]
```

PUT /packages/v2/adjust

```
[{
  "Label": "ABCDEF012345670000010041",
  "Quantity": -2.0,
  "UnitOfMeasure": "Ounces",
  "AdjustmentReason": "Drying",
  "AdjustmentDate": "2015-12-15",
  "ReasonNote": null
}]
```

`Quantity` is the **delta**, signed — not the new total. `AdjustmentReason` must come from `GET /packages/v2/adjust/reasons`.

PUT /packages/v2/finish ▪ unfinish ▪ location ▪ item ▪ note

```
// finish
[{ "Label": "ABCDEF012345670000010041", "ActualDate": "2015-12-15" }]
// unfinish
[{ "Label": "ABCDEF012345670000010041" }]
// location
[{ "Label": "...", "Location": "Storage Closet", "Sublocation": "Shelf 2", "MoveDate":
"2018-03-15" }]
// item
[{ "Label": "...", "Item": "Shake" }]
// note
[{ "PackageLabel": "...", "Note": "Package note here." }]
```

All return an empty body.

POST /sales/v2/receipts

```
{
  "SalesDateTime": "2016-10-04T16:44:53.000",
  "ExternalReceiptNumber": "ABC-1234",
  "SalesCustomerType": "Consumer",
  "PatientLicenseNumber": null, "CaregiverLicenseNumber": null,
  "IdentificationMethod": null, "PatientRegistrationLocationId": null,
  "Transactions": [{
    "PackageLabel": "ABCDEF012345670000010331",
    "Quantity": 1.0, "UnitOfMeasure": "Ounces", "TotalAmount": 9.99,
    "UnitThcPercent": null, "UnitThcContent": null,
    "UnitThcContentUnitOfMeasure": null,
    "UnitWeight": null, "UnitWeightUnitOfMeasure": null,
    "InvoiceNumber": null, "Price": null,
    "ExciseTax": null, "CityTax": null, "CountyTax": null,
    "MunicipalTax": null, "DiscountAmount": null,
    "SubTotal": null, "SalesTax": null, "QrCodes": null
  ]
}
```

PUT is identical plus a top-level "Id" .

POST /items/v2/

```
[{
  "ItemCategory": "Buds", "Name": "Buds Item",
  "GlobalProductName": null, "UnitOfMeasure": "Ounces",
  "Strain": "Spring Hill Kush", "ItemBrand": null,
  "AdministrationMethod": null,
  "UnitCbdPercent": null, "UnitCbdContent": null,
  "UnitCbdContentUnitOfMeasure": null, "UnitCbdContentDose": null,
  "UnitCbdContentDoseUnitOfMeasure": null,
  "UnitThcPercent": null, "UnitThcContent": null,
  "UnitThcContentUnitOfMeasure": null, "UnitThcContentDose": null,
  "UnitThcContentDoseUnitOfMeasure": null,
  "UnitCbdAPercent": null, "UnitCbdAContent": null,
  "UnitCbdAContentUnitOfMeasure": null, "UnitCbdAContentDose": null,
  "UnitCbdAContentDoseUnitOfMeasure": null,
  "UnitThcAPercent": null, "UnitThcAContent": null,
  "UnitThcAContentUnitOfMeasure": null, "UnitThcAContentDose": null,
  "UnitThcAContentDoseUnitOfMeasure": null,
  "UnitVolume": null, "UnitVolumeUnitOfMeasure": null,
  "UnitWeight": null, "UnitWeightUnitOfMeasure": null,
  "ServingSize": null, "SupplyDurationDays": null, "NumberOfDoses": null,
  "PublicIngredients": null, "ItemIngredients": null,
  "Description": null, "Allergens": null,
  "ProductImageFileSystemIds": null, "ProductPhotoDescription": null,
  "LabelImageFileSystemIds": null, "LabelPhotoDescription": null,
  "PackagingImageFileSystemIds": null, "PackagingPhotoDescription": null,
  "ProductPDFFileSystemIds": null, "ProcessingJobCategoryName": null
}]
```

`ItemCategory` must come from `GET /items/v2/categories`. To attach a photo, `POST /items/v2/photo` first and pass the returned ID.

POST /strains/v2/

```
[{
  "Name": "Spring Hill Kush",
  "TestingStatus": "None",
  "ThcLevel": 0.1865, "CbdLevel": 0.1075,
  "IndicaPercentage": 25.0, "SativaPercentage": 75.0
}]
```

`TestingStatus` ∈ `None` / `InHouse` / `ThirdParty`. THC/CBD levels are **fractions, not percentages** (0.1865 = 18.65%).

POST /locations/v2/

```
[{ "Name": "Harvest Location", "LocationTypeName": "Default" }]
```

POST /plantbatches/v2/plantings

```
[{
  "Name": "B. Kush 5-30", "Type": "Clone", "Count": 25,
  "Strain": "Spring Hill Kush",
  "Location": null, "Sublocation": null,
  "PatientLicenseNumber": "X00001",
  "ActualDate": "2022-02-15",
  "SourcePlantBatches": [{ "PlantBatchName": "Red Crush" }]
}]
```

POST /plants/v2/plantings

```
[{
  "PlantLabel": "ABCDEF012345670000010011",
  "PlantBatchName": "Demo Plant Batch 1",
  "PlantBatchType": "Clone", "PlantCount": 3,
  "LocationName": "Plant Location", "SublocationName": "Plant Sublocation",
  "StrainName": "Spring Hill Kush",
  "PatientLicenseNumber": "X00001",
  "ActualDate": "2016-10-18T13:11:03Z"
}]
```

PUT /plants/v2/harvest

```
[{
  "Plant": "ABCDEF012345670000010011",
  "Weight": 100.23, "UnitOfWeight": "Grams",
  "DryingLocation": "Harvest Location",
  "DryingSublocation": "Harvest Sublocation1",
  "HarvestName": "2015-12-15-Harvest Location-H",
  "PatientLicenseNumber": "X00001",
  "ActualDate": "2015-12-15"
}]
```

HarvestName: `null` auto-generates the name. Prefer supplying your own.

POST /harvests/v2/packages

```
[{
  "Tag": "ABCDEF012345670000020201",
  "Location": null, "Sublocation": null,
  "Item": "Buds", "UnitOfWeight": "Grams",
  "PatientLicenseNumber": "X00001", "Note": "This is a note.",
  "IsProductionBatch": false, "ProductionBatchNumber": null,
  "IsTradeSample": false, "IsDonation": false,
  "ProductRequiresRemediation": false, "RemediateProduct": false,
  "RemediationMethodId": null, "RemediationDate": null, "RemediationSteps": null,
  "ProductRequiresDecontamination": false, "DecontaminateProduct": false,
  "DecontaminationDate": null, "DecontaminationSteps": null,
  "ActualDate": "2015-12-15",
  "ExpirationDate": null, "SellByDate": null, "UseByDate": null,
  "Ingredients": [
    { "HarvestId": 2, "HarvestName": null, "Weight": 100.23, "UnitOfWeight": "Grams" },
    { "HarvestId": null, "HarvestName": "2018-04-03-Harvest Location-M",
      "Weight": 25.1, "UnitOfWeight": "Grams" }
  ],
  "ProcessingJobTypeId": 1, "LabTestStageId": 2,
  "RequiredLabTestBatches": []
}]
```

Ingredients accept **either** `HarvestId` **or** `HarvestName` — the name-matching rule from §2.10.

POST /processing/v2/start

```
[{
  "JobName": "Job 1", "JobType": "Infusing",
  "CountUnitOfMeasure": "Each",
  "VolumeUnitOfMeasure": "Fluid Ounces",
  "WeightUnitOfMeasure": "Ounces",
  "Packages": [
    { "Label": "ABCDEF012345670000010042", "Quantity": 10.0, "UnitOfMeasure": "Grams" }
  ],
  "StartDate": "0001-01-01T00:00:00Z"
}]
```

POST /processing/v2/createpackages

```
[{
  "JobName": "Job 1", "Tag": "ABCDEFG11232131",
  "Location": null, "Sublocation": null, "Item": null,
  "Quantity": 12.0, "UnitOfMeasure": "Grams",
  "IsFinishedGood": false, "PatientLicenseNumber": null, "Note": null,
  "ProductionBatchNumber": null,
  "FinishProcessingJob": false, "FinishDate": null,
  "WasteCountQuantity": null, "WasteCountUnitOfMeasureName": null,
  "WasteVolumeQuantity": null, "WasteVolumeUnitOfMeasureName": null,
  "WasteWeightQuantity": null, "WasteWeightUnitOfMeasureName": null,
  "FinishNote": null, "PackageDate": null,
  "ExpirationDate": null, "SellByDate": null, "UseByDate": null
}]
```

POST /labtests/v2/record

```
[{
  "Label": "ABCDEF012345670000000001",
  "ResultDate": "2015-12-15T00:00:00Z",
  "DocumentFileName": "Lab Results 20151215.pdf",
  "DocumentFileBase64": "File encoded in Base64==",
  "Results": [
    { "LabTestTypeName": "THC", "Quantity": 100.2345, "Passed": true, "Notes": "" }
  ]
}]
```

PDF only, 5 MB maximum. `Label` is a package label. `LabTestTypeName` must come from `GET /labtests/v2/types`.

POST /transfers/v2/external/incoming

```
[{
  "ShipperLicenseNumber": "123-ABC",
  "ShipperName": "Lofty Med-Cultivation B",
  "ShipperMainPhoneNumber": "123-456-7890",
  "ShipperAddress1": "123 Real Street", "ShipperAddress2": null,
  "ShipperAddressCity": "Somewhere", "ShipperAddressState": "CO",
  "ShipperAddressPostalCode": null,
  "TransporterFacilityLicenseNumber": null,
  "DriverOccupationalLicenseNumber": null, "DriverName": null,
  "DriverLicenseNumber": null, "PhoneNumberForQuestions": null,
  "VehicleMake": null, "VehicleModel": null,
  "VehicleLicensePlateNumber": null, "VehicleRegistrationNumber": null,
  "Destinations": [{
    "RecipientLicenseNumber": "123-XYZ",
    "InvoiceNumber": "INV-001-0236521",
    "TransferTypeName": "Transfer",
    "PlannedRoute": "I will drive down the road to the place.",
    "EstimatedDepartureDateTime": "2018-03-06T09:15:00.000",
    "EstimatedArrivalDateTime": "2018-03-06T12:24:00.000",
    "GrossWeight": null, "GrossUnitOfWeightId": null,
    "Transporters": [{
      "TransporterFacilityLicenseNumber": "123-ABC",
      "DriverOccupationalLicenseNumber": "50",
      "DriverName": "X", "DriverLicenseNumber": "5",
      "DriverLayoverLeg": "FromAndToLayover",
      "PhoneNumberForQuestions": "18005555555",
      "VehicleMake": "X", "VehicleModel": "X",
      "VehicleLicensePlateNumber": "X", "VehicleRegistrationNumber": null,
      "IsLayover": false,
      "EstimatedDepartureDateTime": "2018-03-06T12:00:00.000",
      "EstimatedArrivalDateTime": "2018-03-06T21:00:00.000",
      "TransporterDetails": [ /* per-leg driver/vehicle */ ]
    }],
    "Packages": [ /* tags being moved */ ]
  }]
}]
```

TransferTypeName must come from [GET /transfers/v2/types](#).

POST /transfers/v2/templates/outgoing

Same shape as external incoming, minus the **Shipper*** block, plus a top-level **"Name"** and **"PaymentTermDays"** on each destination.

End of guide.